

به نام خدا

پروژه درس مخابرات بی سیم

پریسا محمدی

شماره دانشجویی: 810101509

خلاصه خیلی کوتاه کار

در این پروژه یک زنجیره کامل OFDM پیاده سازی شد: تولید بیت تصادفی، مدولاسیون QPSK، فریم بندی، مدولاسیون تفاضلی DPSK، تخصیص حامل ها، IFFT، اضافه کردن پیشوند گردشی، ارسال از کانال، و در نهایت بازیابی داده در گیرنده. علاوه بر حالت AWGN، کانال رایلی هم بررسی شد. یک بار رایلی با مدل ساده تر اجرا شد و بعد در نسخه دوم برای این که مدل گیرنده واقعی تر باشد، برای رایلی یک one-tap MMSE equalizer در حوزه فرکانس در نظر گرفته شد.

1 هدف پروژه و چک لیست خواسته ها

هدف تمرین، شبیه سازی یک سیستم OFDM برای ارسال یک پیام باینری شامل 10^7 بیت بود. پیام ابتدا به صورت تصادفی تولید می شود، بعد با QPSK مدوله می شود و سپس به فریم های OFDM تبدیل می شود. در فرستنده، طبق ساختار صورت پروژه، بین فریم ها Frame Guard قرار داده شده و در ابتدا و انتهای کل سیگنال هم Header اضافه شده است. در گیرنده هم همین مراحل به صورت معکوس انجام می شود تا بیت ها تخمین زده شوند.

برای این که مطمئن باشم همه خواسته های صورت پروژه در گزارش آمده اند، جدول زیر را به عنوان چک لیست آورده ام.

جدول ۱: چکلیست قسمت‌های خواسته شده در پروژه

خواسته پروژه	کاری که در کد انجام شد	نتیجه در گزارش
تولید پیام 10^7 بیتی و مدولاسیون QPSK	تولید بیت با randi و تبدیل هر دو بیت به یک سمبل QPSK	بخش‌های ۲ و ۳
فریم‌بندی، DPSK، IFFT و اضافه کردن CP	ساخت فریم‌ها با ofdm_tx و build_one_ofdm_frame	بخش ۳
بخش الف: کانال AWGN با SNR برابر 20 dB ، $N_{FFT} = 20$ و $n_c = 400$ و 1024	اجرای کامل سیستم و محاسبه BER	بخش ۶
گزارش تعداد فریم‌ها	محاسبه از روی ظرفیت هر فریم	بخش ۲ و ۶
توضیح بلوک‌های OFDM	گیرنده توضیح حذف Header/Guard، حذف CP، FFT، استخراج حامل‌ها، DPSK و تصمیم‌گیری QPSK	بخش ۲
بخش ب: نمودار BER بر حسب SNR برای AWGN	اجرای SNR از ۰ تا ۲۰ دسی‌بل	بخش ۷ و شکل‌های مربوطه
بخش ج: کانال رایلی با واریانس واحد	اجرای رایلی در کد اول و سپس نسخه اصلاح شده با MMSE equalizer	بخش‌های ۸ و ۹
بخش امتیازی: تاخیر نامعلوم و تشخیص Header	اضافه کردن نمونه تصادفی قبل از Header و پیدا کردن شروع با correlation	بخش ۱۱

۲ پارامترهای اصلی و محاسبات اولیه

پارامترهای اصلی پروژه مطابق صورت تمرین تنظیم شدند. مقادیر مهم در جدول زیر آمده‌اند. اعداد داخل جدول عمده‌اً با دستور \lr نوشته شده‌اند تا در متن فارسی، از چپ به راست و بدون به هم ریختن ممیز نمایش داده شوند.

جدول ۲: پارامترهای اصلی شبیه سازی

پارامتر	مقدار
تعداد بیت های پیام	10^7
مدولاسیون	QPSK
تعداد حامل ها	$n_c = 400$
طول FFT و IFFT	$N_{FFT} = 1024$
طول CP	256 نمونه
بازه SNR برای نمودارها	0 : 2 : 20 dB
SNR بخش الف	20 dB
طول Header	8 برابر طول یک فریم
طول Frame Guard	برابر طول یک فریم

طبق صورت پروژه، تعداد سمبل ها در هر فریم برای هر حامل به صورت زیر تعریف شده است:

$$\text{sfc} = \left\lceil \frac{2^{13}}{n_c} \right\rceil.$$

برای $n_c = 400$ داریم:

$$\text{sfc} = \left\lceil \frac{8192}{400} \right\rceil = 21.$$

چون در QPSK هر سمبل دو بیت حمل می کند، تعداد سمبل های QPSK برای پیام 10^7 بیتی برابر است با:

$$\frac{10^7}{2} = 5,000,000.$$

ظرفیت هر فریم هم برابر است با:

$$n_c \times \text{sfc} = 400 \times 21 = 8400.$$

پس تعداد فریم ها می شود:

$$\left\lceil \frac{5,000,000}{8400} \right\rceil = 596.$$

همچنین تعداد سمبل های صفر اضافه شده به انتهای داده ها برابر است با:

$$596 \times 8400 - 5,000,000 = 6400.$$

پس دو عدد مهمی که کد چاپ کرده، یعنی 596 فریم و 6400 سمبل padding، با محاسبه دستی کاملاً یکی هستند.

خروجی پارامترهای اصلی در اجرای نهایی

System parameters:

sfc = 21

CP length = 256

OFDM symbols per frame = 22

Time samples per frame = 28160

Number of frames = 596

Number of padded QPSK symbols = 6400

Useful signal power = 0.000762804

طول هر سمبل OFDM بعد از اضافه شدن CP برابر است با:

$$1024 + 256 = 1280.$$

از آن جا که قبل از DPSK یک سطر مرجع هم اضافه می شود، تعداد ستون های زمانی هر فریم برابر $sfc+1 = 22$ است. بنابراین طول زمانی هر فریم دقیقاً می شود:

$$22 \times 1280 = 28160.$$

این عدد هم با خروجی کد یکی است.

3 توضیح پیاده سازی فرستنده و کد مربوط به آن

در این قسمت خود کد را کامل داخل گزارش نیاوردم، چون فایل های m. جداگانه تحویل داده می شوند. ولی منطق کد و کار هر تابع را توضیح می دهم تا معلوم باشد هر بخش دقیقاً چه کاری انجام داده است.

1.3 تولید بیت و مدولاسیون QPSK

در ابتدای کد، با دستور randi یک بردار تصادفی از صفر و یک ساخته شد. سپس تابع bits_to_qpsk_index هر دو بیت را به یک عدد بین 0 تا 3 تبدیل می کند. نگاشت استفاده شده ساده و مستقیم است:

$$00 \rightarrow 0, \quad 01 \rightarrow 1, \quad 10 \rightarrow 2, \quad 11 \rightarrow 3.$$

بعداً در انتهای گیرنده، تابع qpsk_index_to_bits همین کار را برعکس انجام می دهد و اندیس های تخمینی را به بیت تبدیل می کند.

2.3 فریم بندی و اضافه کردن صفرهای انتهایی

ظرفیت هر فریم برابر 8400 سمبل است. اگر تعداد سمبل های QPSK دقیقاً مضرب این عدد نباشد، کد به آخر داده ها صفر اضافه می کند تا فریم آخر هم کامل شود. این دقیقاً همان چیزی است که در صورت پروژه هم خواسته شده بود. برای داده 10^7 بیتی، تعداد سمبل های اضافه شده برابر 6400 شد.

3.3 مدولاسیون تفاضلی DPSK

در تابع `build_one_ofdm_frame`، اول یک سطر مرجع تصادفی به بالای ماتریس داده اضافه می‌شود. سپس برای هر حامل، DPSK در راستای سطرها انجام می‌شود. رابطه پیاده‌سازی شده به صورت زیر است:

$$d_{r,c} = (a_{r,c} + d_{r-1,c}) \bmod 4.$$

در این رابطه، $a_{r,c}$ اندیس سمبل اصلی و $d_{r,c}$ اندیس بعد از DPSK است. مزیت این کار این است که گیرنده به جای استفاده مستقیم از فاز مطلق، از اختلاف فاز دو سمبل پشت سر هم استفاده می‌کند. به همین دلیل، این روش برای حالت‌هایی که فاز مطلق دقیقاً معلوم نیست، مناسب‌تر است.

4.3 تخصیص حامل‌ها، تقارن مزدوج و IFFT

بعد از DPSK، اندیس‌های 0 تا 3 به فازهای 0° ، 90° ، 180° و 270° تبدیل شدند. سپس داده‌ها در بین‌های مثبت فرکانسی قرار گرفتند. برای این‌که خروجی IFFT حقیقی باشد، در سمت مقابل طیف، مزدوج قرینه داده‌ها قرار داده شد:

$$X(N - k) = X^*(k).$$

در نتیجه بعد از گرفتن IFFT، قسمت حقیقی سیگنال به عنوان نمونه‌های زمانی فریم ذخیره شد.

5.3 اضافه کردن CP، Header و Frame Guard

برای ساخت CP، 256 نمونه انتهایی هر سمبل OFDM به ابتدای همان سمبل اضافه شد. سپس کل فریم به صورت سریالی نوشته شد. بین فریم‌های متوالی یک Frame Guard با طول یک فریم گذاشته شد. همچنین در ابتدای کل سیگنال و در انتهای آن، یک Header با طول 8 برابر طول یک فریم اضافه شد. این Header در بخش امتیازی برای پیدا کردن ابتدای سیگنال استفاده شده است.

جدول ۳: خلاصه نقش تابع‌های اصلی کد

تابع	توضیح
ofdm_tx	تبدیل کل بیت‌ها به سیگنال ارسالی، شامل QPSK، فریم‌بندی، DPSK، IFFT، CP، Header و Guard.
build_one_ofdm_frame	ساخت یک فریم OFDM از ماتریس داده، اضافه کردن سطر مرجع، انجام DPSK و گرفتن IFFT.
add_awgn	اضافه کردن نویز سفید گاوسی با توان متناسب با SNR.
ofdm_rx	گیرنده اصلی برای حالت AWGN و حالت سنکرون.
decode_one_ofdm_frame	دیکد کردن یک فریم: حذف CP، گرفتن FFT، استخراج حامل‌ها و DPSK demodulation.
detect_header_start	پیدا کردن شروع Header در بخش امتیازی با normal-ized correlation.

4 توضیح بلوک‌های گیرنده OFDM

گیرنده تقریباً همان مسیر فرستنده را برعکس می‌رود. اول Header و Frame Guard کنار گذاشته می‌شوند و هر فریم اصلی جدا می‌شود. سپس فریم از حالت سریالی به ماتریسی برمی‌گردد. بعد از آن، CP حذف می‌شود، چون CP فقط برای حفظ ساختار چرخشی و مقابله با اثر کانال اضافه شده و خودش اطلاعات جدیدی ندارد.

بعد از حذف CP، از هر ستون FFT گرفته می‌شود تا از حوزه زمان دوباره به حوزه فرکانس برگردیم. سپس فقط همان زیرحامل‌هایی برداشته می‌شوند که در فرستنده داده روی آن‌ها قرار گرفته بود. در ادامه DPSK با استفاده از اختلاف فاز دو سطر متوالی آشکارسازی می‌شود:

$$z_{r,c} = \hat{D}_{r,c} \hat{D}_{r-1,c}^*$$

فاز $z_{r,c}$ به نزدیک‌ترین فاز QPSK گرد می‌شود. در نهایت اندیس‌های به‌دست‌آمده دوباره به بیت تبدیل می‌شوند و بیت‌های اضافه‌شده ناشی از padding حذف می‌شوند. معیار نهایی هم BER است:

$$BER = \frac{\text{تعداد بیت‌های اشتباه}}{\text{تعداد کل بیت‌های ارسال شده}}$$

5 بخش الف: کانال AWGN در 20 dB

در بخش الف، کانال AWGN با میانگین صفر در نظر گرفته شد. برای ساخت نویز، ابتدا SNR دسی‌بل به مقدار خطی تبدیل شد:

$$SNR_{lin} = 10^{SNR_{dB}/10}$$

سپس واریانس نویز از رابطه زیر محاسبه شد:

$$\sigma_n^2 = \frac{P_x}{\text{SNR}_{\text{lin}}},$$

که P_x توان متوسط سیگنال مفید است. این رابطه همان ایده‌ای است که صورت پروژه خواسته بود؛ یعنی با تغییر واریانس نویز، مقدار SNR تغییر کند.

خروجی بخش الف

Part A: AWGN, SNR = 20.0 dB

BER Part A = 0

Number of frames Part A = 596

از نظر تئوری این خروجی قابل قبول است. در SNR برابر 20 dB، نویز نسبت به توان سیگنال خیلی کم است و برای QPSK روی کانال AWGN انتظار داریم خطا بسیار کم شود. چون BER در شبیه‌سازی به صورت تجربی حساب شده، وقتی در کل 10^7 بیت هیچ خطایی دیده نشود، مقدار BER صفر چاپ می‌شود. بنابراین صفر شدن BER به معنی این نیست که احتمال خطای تئوری دقیقاً صفر است؛ یعنی در این اجرای محدود، خطایی مشاهده نشده است.

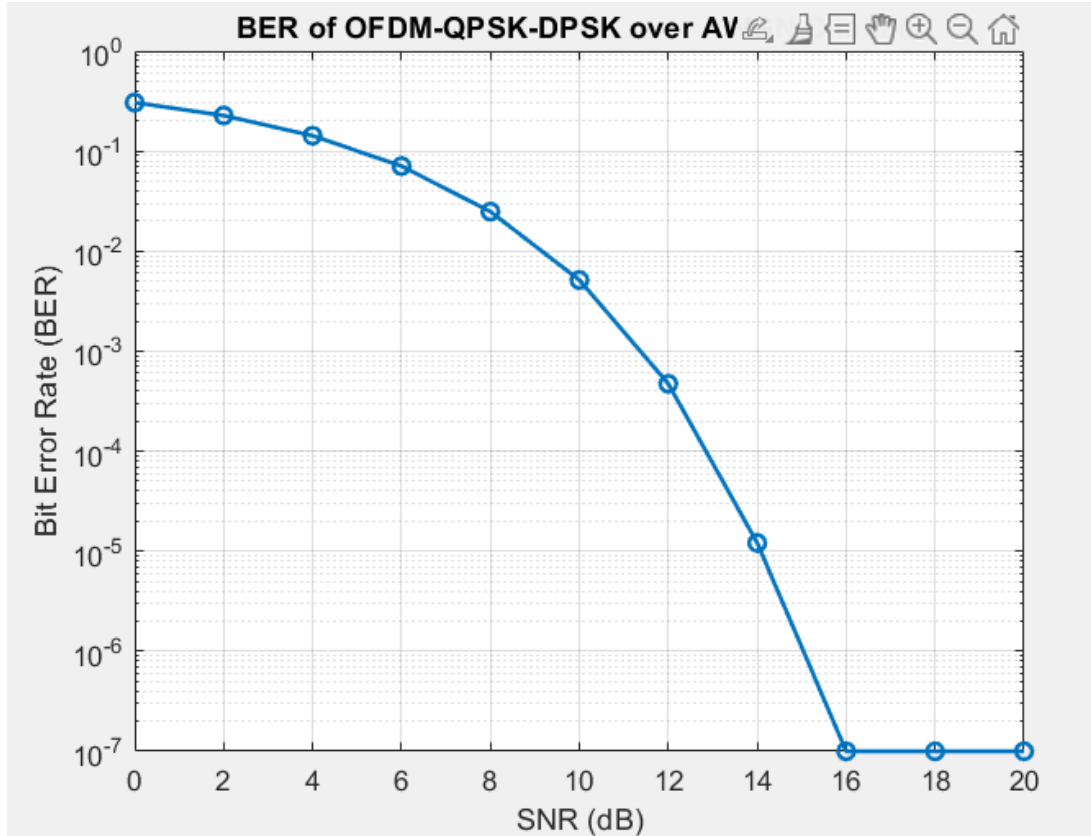
6 بخش ب: نمودار BER برای کانال AWGN

برای بخش ب، SNR از 0 تا 20 دسی‌بل با گام 2 تغییر داده شد. برای هر مقدار SNR، سیگنال ارسالی ثابت بود و فقط توان نویز عوض شد. جدول زیر خروجی عددی AWGN را نشان می‌دهد.

جدول ۴: نتایج BER برای کانال AWGN

BER	SNR (dB)
0.305619	0
0.226811	2
0.142511	4
0.0708073	6
0.0248278	8
0.0051443	10
0.0004719	12
1.21×10^{-5}	14
1×10^{-7}	16
0	18
0	20

روند جدول و نمودار کاملاً همان چیزی است که از نظر تئوری انتظار داریم. وقتی SNR کم است، نویز قوی است و BER زیاد می‌شود. با افزایش SNR، تصمیم‌گیری QPSK دقیق‌تر می‌شود و خطا کاهش پیدا می‌کند. در 18 و 20 دسی‌بل خطای مشاهده‌شده صفر شده است. برای رسم روی محور لگاریتمی، در کد مقادیر صفر با مقدار خیلی کوچکی مثل $1/N_{bits}$ جایگزین شدند تا نقطه روی نمودار دیده شود.



شکل ۱: نمودار BER بر حسب SNR برای کانال AWGN در کد اول

7 بخش ج در کد اول: کانال رایلی ساده

در کد اول، کانال رایلی به صورت فیدینگ تخت برای هر فریم مدل شد. برای هر فریم یک ضریب تصادفی رایلی تولید شد و کل فریم در آن ضریب ضرب شد. ضریب طوری ساخته شد که توان متوسط آن واحد باشد:

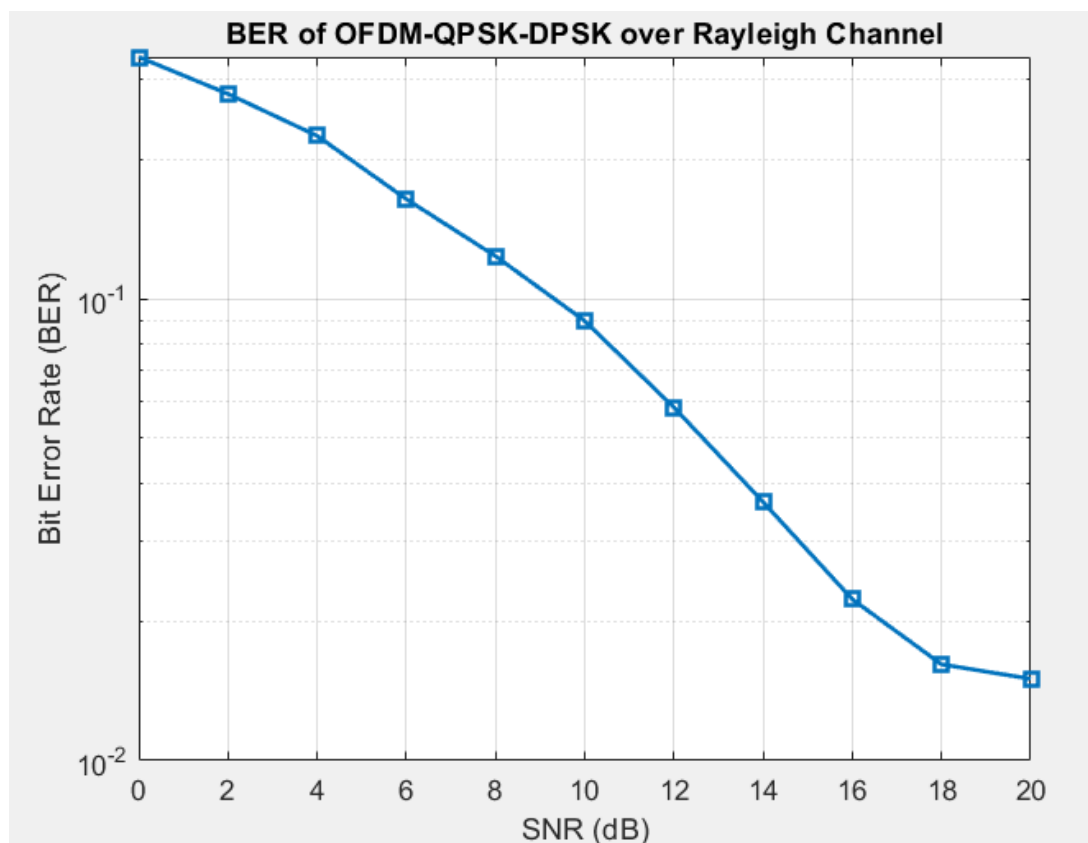
$$\mathbb{E}\{h^2\} = 1.$$

این مدل با جمله «کانال رایلی با واریانس واحد» سازگار است، ولی در این نسخه equalizer صریحی برای جبران کانال قرار داده نشده بود. بنابراین اگر برای یک فریم مقدار h کوچک شود، آن فریم وارد حالت deep fade می‌شود و حتی اگر SNR کلی زیاد باشد، احتمال خطا بالا می‌رود.

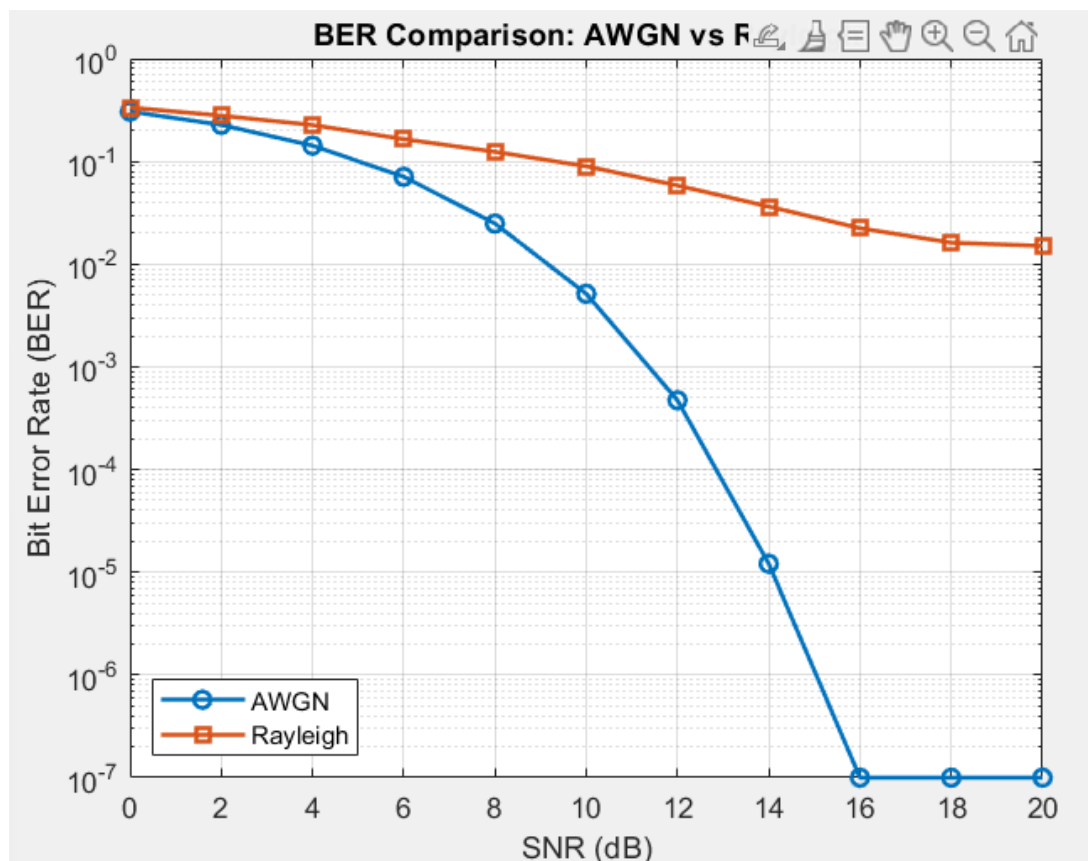
جدول ۵: نتایج BER کانال رایلی در کد اول، بدون equalizer صریح

BER	SNR (dB)
0.335054	0
0.278762	2
0.226051	4
0.16483	6
0.123993	8
0.0897284	10
0.0582675	12
0.0363479	14
0.0223651	16
0.0161642	18
0.0150368	20

این خروجی از نظر کلی درست است، چون با افزایش SNR مقدار BER کم شده است. همچنین رایلی از AWGN بدتر است، چون علاوه بر نویز، فیدینگ هم داریم. نکته‌ای که در خروجی اول دیده شد این بود که در 20 dB مقدار BER هنوز حدود 0.015 بود. این برای مدلی که کانال رایلی را جبران نمی‌کند قابل دفاع است، اما برای یک گیرنده OFDM عملی‌تر، بهتر است یک equalizer هم در نظر گرفته شود. به همین دلیل نسخه دوم کد نوشته شد.



شکل ۲: نمودار BER برای کانال رایلی در کد اول



شکل ۳: مقایسه خروجی AWGN و رایلی در کد اول

8 کد دوم: رایلی با one-tap MMSE equalizer

در نسخه دوم، بخش‌های تولید داده، فرستنده، AWGN و بخش امتیازی همان ساختار قبلی را دارند. تغییر اصلی فقط در بخش کانال رایلی و گیرنده رایلی است. در OFDM، بعد از FFT، اثر کانال روی هر زیرحامل معمولاً به صورت یک ضرب ساده دیده می‌شود. بنابراین مدل زیر برای هر زیرحامل در نظر گرفته شد:

$$Y_k[m] = H_k[m]X_k[m] + W_k[m].$$

در این رابطه، $X_k[m]$ سمبل ارسالی روی زیرحامل، $H_k[m]$ پاسخ کانال و $W_k[m]$ نویز است. برای کانال رایلی با توان واحد فرض شد:

$$H_k[m] \sim \mathcal{CN}(0, 1), \quad \mathbb{E}\{|H_k[m]|^2\} = 1.$$

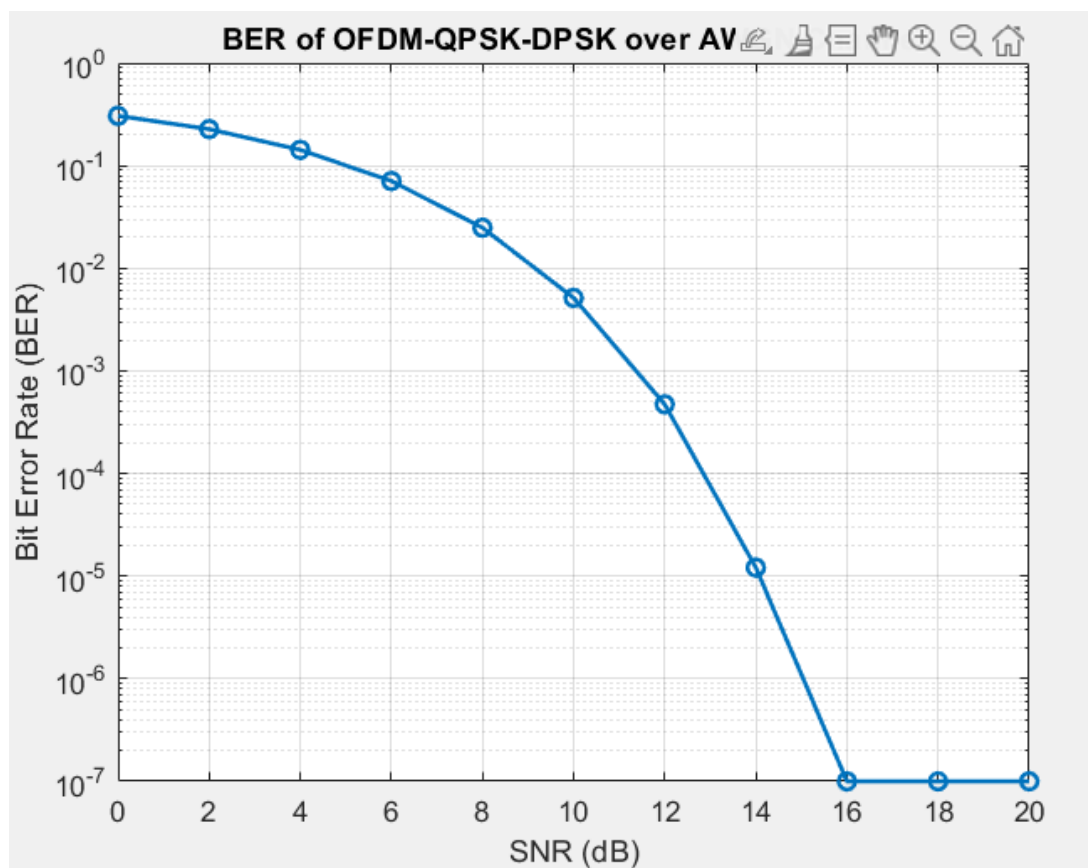
پس دامنه $|H_k[m]|$ رفتار رایلی دارد و شرط واریانس [توان واحد هم رعایت شده است. این فرض صورت پروژه را نقض نمی‌کند، چون کانال همچنان رایلی با توان متوسط واحد است. فقط برای گیرنده فرض شده است که پاسخ کانال قابل تخمین است؛ مثلاً از روی Header، سطر مرجع، یا یک مرحله آموزشی ساده. بعد از FFT و استخراج حامل‌ها، روی هر زیرحامل MMSE equalizer زیر اعمال شد:

$$\hat{X}_k[m] = \frac{H_k^*[m]}{|H_k[m]|^2 + \sigma_W^2/E_s} Y_k[m].$$

در کد، چون سمبل‌های QPSK دامنه واحد دارند، E_s تقریباً برابر 1 در نظر گرفته شده است. دلیل استفاده از MMSE به جای ZF این بود که وقتی $|H|$ خیلی کوچک می‌شود، ZF نویز را خیلی زیاد تقویت می‌کند، ولی MMSE به خاطر جمله نویز در مخرج، این مشکل را کمتر می‌کند.

جدول 6: تابع‌های اضافه‌شده یا تغییرکرده در نسخه دوم کد

تابع	نقش در کد دوم
بخش تولید کانال رایلی و MMSE rayleigh_...	ساخت کانال رایلی در حوزه فرکانس، اضافه کردن نویز، ذخیره پاسخ کانال و سپس فراخوانی گیرنده رایلی با CSI.
گیرنده رایلی با MMSE ofdm_rx_...	گیرنده مخصوص رایلی که برای هر فریم، پاسخ کانال همان فریم را هم به دیکدر می‌دهد.
دیکدر فریم با equalizer decode_one_...	حذف CP، گرفتن FFT، اعمال MMSE equalizer روی حامل‌ها و سپس demodulation DPSK.

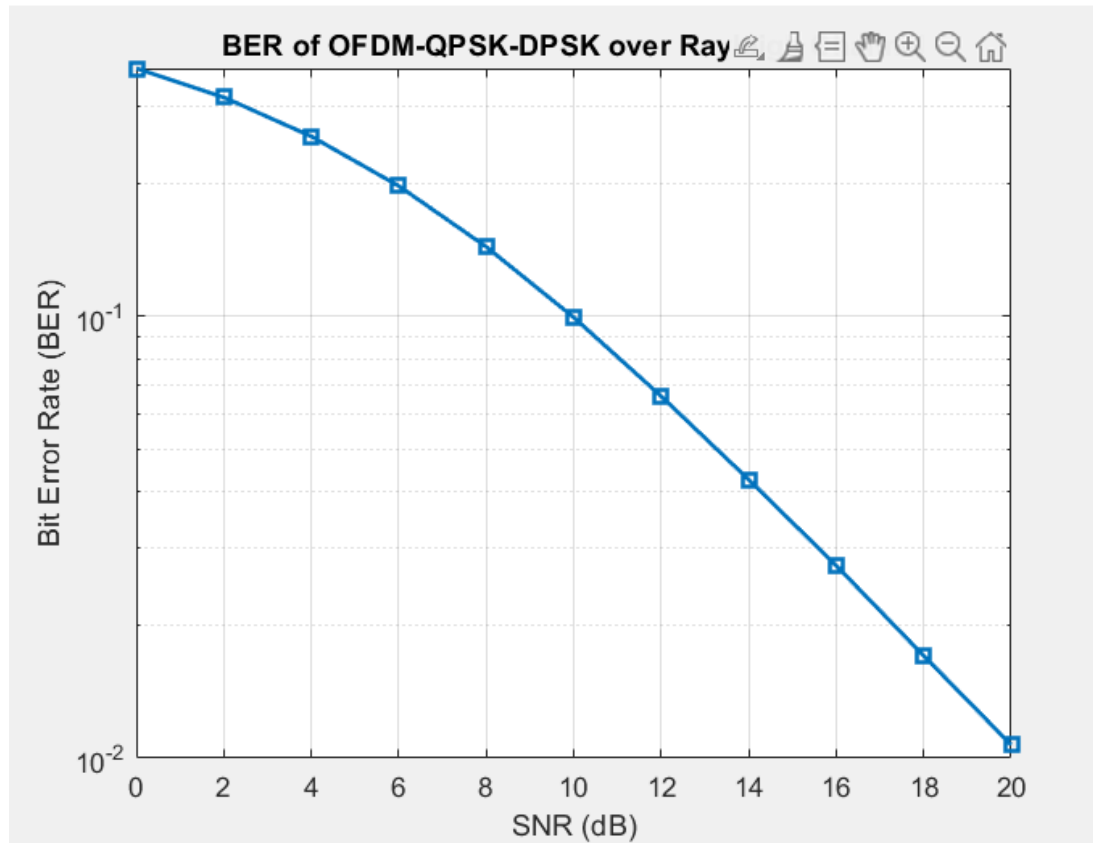


شکل ۴: نمودار BER برای AWGN در کد دوم؛ این بخش مثل کد اول است

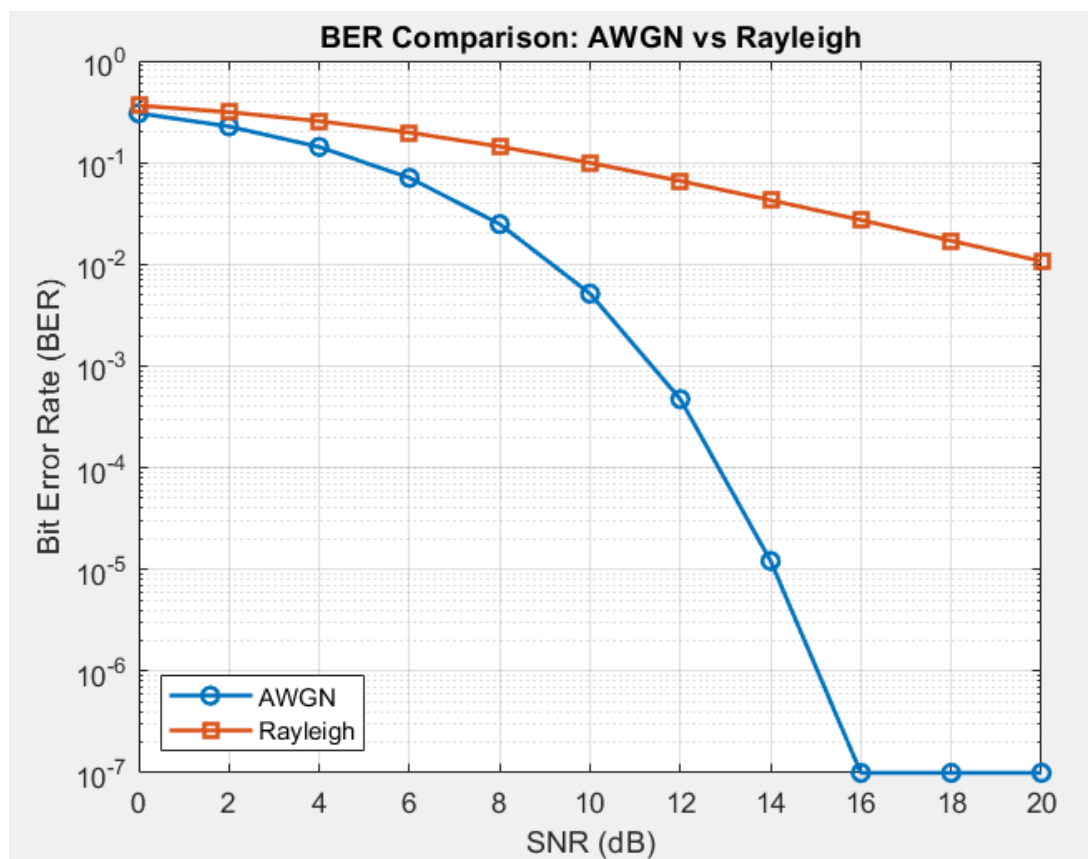
جدول ۴: نتایج BER کانال رایلی در کد دوم، با MMSE equalizer

BER	SNR (dB)
0.364341	0
0.313503	2
0.255704	4
0.197461	6
0.143765	8
0.0995132	10
0.065963	12
0.0427175	14
0.0272917	16
0.0170534	18
0.0107161	20

در نسخه دوم، منحنی راییلی باز هم از AWGN بالاتر است که طبیعی است، چون راییلی به خاطر deep fade سخت‌تر از AWGN است. اما نکته مثبت این است که در 20 dB مقدار BER به حدود 0.0107 رسیده است، در حالی که در کد اول حدود 0.0150 بود. یعنی در SNR بالا، فرض equalizer نتیجه راییلی را کمی بهتر و از نظر گیرنده OFDM قابل قبول‌تر کرده است. در SNRهای پایین، ممکن است به خاطر تفاوت مدل کانال و نویز مختلط، اعداد دقیقاً کمتر از کد اول نشوند، ولی روند کلی درست است: با افزایش SNR، BER پایین می‌آید و منحنی راییلی از AWGN بدتر می‌ماند.



شکل ۵: نمودار BER راییلی در کد دوم با one-tap MMSE equalizer



شکل ۶: مقایسه AWGN و رایلی در کد دوم

9 مقایسه کلی خروجی‌های کد اول و کد دوم

در هر دو نسخه، قسمت‌های اصلی سیستم یکی هستند و خروجی‌های مربوط به AWGN هم عملاً یکسان‌اند. تفاوت اصلی در مدل رایلی است. در کد اول رایلی به صورت فیدینگ ساده و بدون جبران صریح کانال اعمال شد. این مدل برای نشان دادن اثر فیدینگ خوب است، اما در SNR بالا خطا را نسبتاً زیاد نگه می‌دارد. در کد دوم، پاسخ کانال در حوزه فرکانس تولید شد و گیرنده با فرض داشتن تخمین کانال، یک MMSE equalizer یک‌تپی اعمال کرد.

جدول ۸: مقایسه کوتاه دو مدل رایلی

ویژگی	کد اول	کد دوم
مدل رایلی	فیدینگ تخت برای هر فریم	کانال فرکانسی روی زیرحاملها
جبران کانال	ندارد	MMSE one-tap equalizer
فرض گیرنده	فقط گیرنده پایه OFDM	گیرنده با تخمین کانال
BER در 20 dB	0.0150368	0.0107161
تفسیر	ساده و قابل دفاع	برای OFDM عملی تر و کامل تر

بنابراین برای گزارش نهایی، نسخه دوم از نظر مخابراتی کامل تر است، چون استفاده از equalizer در گیرنده OFDM فرض عجیبی نیست. با این حال، خروجی‌های کد اول هم آورده شدند تا معلوم باشد قبل از اضافه کردن equalizer، رفتار رایلی چگونه بوده است.

10 بخش امتیازی: آشکارسازی Header در حضور تاخیر نامعلوم

در بخش امتیازی، قبل از Header یک دنباله تصادفی با طول تصادفی اضافه شد. گیرنده مقدار این تاخیر را نمی‌داند و فقط می‌داند که در ابتدای سیگنال باید دنبال Header بگردد. برای این کار، تابع detect_header_start از normalized correlation استفاده می‌کند. ایده این است که Header در گیرنده معلوم است، پس آن را روی بخش ابتدایی سیگنال دریافتی می‌لغزانیم و نقطه‌ای را که بیشترین شباهت را دارد، به عنوان شروع Header انتخاب می‌کنیم.

خروجی بخش امتیازی در کد اول:

کد اول - تشخیص Header

```
True delay = 21304 samples
Detected start index = 21305
Detected delay = 21304 samples
BER after synchronization = 0
```

خروجی بخش امتیازی در کد دوم:

کد دوم - تشخیص Header

```
True delay = 3434 samples
Detected start index = 3435
Detected delay = 3434 samples
BER after synchronization = 0
```

این خروجی‌ها نشان می‌دهند که آشکارسازی درست انجام شده است. اختلاف یک واحدی بین Detected start index و Detected delay به خاطر اندیس‌گذاری متلب است؛ چون در متلب اندیس‌ها از 1 شروع می‌شوند. مثلاً اگر قبل از Header تعداد 3434 نمونه اضافه شده باشد، اولین نمونه Header در اندیس 3435 قرار می‌گیرد. بعد از پیدا شدن شروع سیگنال، گیرنده از همان نقطه سیگنال را جدا کرده و بخش الف را دوباره اجرا کرده است. صفر شدن BER بعد از هم‌زمان‌سازی نشان می‌دهد که بلوک Header detection درست عمل کرده است.

11 جمع‌بندی نهایی

در این پروژه، زنجیره کامل OFDM پیاده‌سازی و بررسی شد. مقدار sfc، طول CP، تعداد فریم‌ها و تعداد سمبل‌های اضافه‌شده به انتهای داده‌ها با محاسبات تئوری سازگار بودند. در بخش AWGN، همان‌طور که انتظار داشتیم، با افزایش SNR مقدار BER کم شد و در SNRهای بالا خطای مشاهده‌شده به صفر رسید. در بخش رایلی، عملکرد از AWGN ضعیف‌تر بود، چون کانال رایلی علاوه بر نویز، افت دامنه و deep fade هم ایجاد می‌کند.

کد اول یک مدل ساده‌تر از رایلی را نشان داد. کد دوم با اضافه کردن one-tap MMSE equalizer، مدل گیرنده را واقعی‌تر کرد و در SNR بالا خروجی رایلی را بهتر کرد. در نهایت، بخش امتیازی هم با استفاده از همبستگی با Header توانست ابتدای سیگنال را در حضور تاخیر نامعلوم پیدا کند و بعد از هم‌زمان‌سازی، BER دوباره صفر شد.